

FERO: Efficient Deep Reinforcement Learning based UAV Obstacle Avoidance at the Edge

PATRICK McENROE ^{1,2} (Associate Member, IEEE), SHEN WANG ¹ (Senior Member, IEEE),
AND MADHUSANKA LIYANAGE ¹ (Senior Member, IEEE)

¹Network Softwarization and Security Labs (NetsLab), School of Computer Science, University College Dublin, D04C1P1 Dublin, Ireland

²SFI Centre for Research Training in Machine Learning (ML Labs), School of Computer Science, University College Dublin, D04C1P1 Dublin, Ireland

CORRESPONDING AUTHOR: PATRICK McENROE (e-mail: patrick.mc-enroe@ucdconnect.ie).

This work was supported by the Science Foundation Ireland (SFI) Centre for Research Training in Machine Learning under Grant 18/CRT/6183.

ABSTRACT With the expanding use of unmanned aerial vehicles (UAVs) across various fields, efficient obstacle avoidance has become increasingly crucial. This UAV obstacle avoidance can be achieved through deep reinforcement learning (DRL) algorithms deployed directly on-device (i.e., at the edge). However, practical deployment is constrained by high training time and high inference latency. In this paper, we propose methods to improve DRL-based UAV obstacle avoidance efficiency through improving both training efficiency and inference latency. To reduce inference latency, we employ input dimension reduction, streamlining the state representation to enable faster decision-making. For training time reduction, we leverage transfer learning, allowing the obstacle avoidance models to rapidly adapt to new environments without starting from scratch. To show the generalizability of our methods, we applied them to a discrete action space dueling double deep Q-network (D3QN) model and a continuous action space soft actor critic (SAC) model. Inference results are evaluated on both an NVIDIA Jetson Nano edge device and a NVIDIA Jetson Orin Nano edge device and we propose a combined method called FERO which combines state space reduction, transfer learning, and conversion to TensorRT for optimum deployment on NVIDIA Jetson devices. For our individual methods and combined method, we demonstrate reductions in training and inference times with minimal compromise in obstacle avoidance performance.

INDEX TERMS Deep Reinforcement Learning (DRL), efficient training, inference latency, obstacle avoidance, transfer learning, UAV, edge AI.

I. INTRODUCTION

Unmanned aerial vehicles (UAVs), also known as drones, can be deployed for numerous applications such as delivery systems and infrastructure inspection. Many of these applications require the UAVs to have the ability to autonomously navigate where obstacle avoidance is crucial [1]. This UAV obstacle avoidance can be deployed at the edge (e.g., on-device or at edge servers) or using cloud servers. We explore an on-device (i.e., on-UAV) edge implementation.

A well researched method of UAV obstacle avoidance is Deep Reinforcement Learning (DRL). Reinforcement Learning (RL) is a subset of machine learning where an agent interacts with an environment until it can determine which of its actions will result in the most reward [2]. DRL combines

neural networks with RL such that large state spaces and large action spaces can be efficiently handled.

DRL-based UAV obstacle avoidance has various limiting factors that still require further work by the research community. An important limiting factor of DRL-based UAV obstacle avoidance (and DRL in general) is the time it takes to train an effective model. In general, due to its trial-and-error nature, DRL training is more time consuming than supervised training [3]. It can take a large amount of episodes for a particular DRL model to reach an acceptable performance [4]. Another limiting factor of DRL-based UAV obstacle avoidance is inference latency. In the context of UAV obstacle avoidance, inference latency is the time it takes a UAV to process sensor input (e.g., camera images) and decide on an action. UAVs

will crash if inference latency is too high [5]. Our paper will explore methods to improve the efficiency of DRL-based UAV obstacle avoidance models through reducing training time and inference latency.

For a similar DRL UAV obstacle avoidance problem, one of our previous works looked at reducing training time and another of our previous works examined reducing inference latency. The first of these works (McEnroe et al. [6]) examined reducing training time for a DRL UAV obstacle avoidance problem by splitting large environments into multiple smaller environments. We trained separate models for each of these smaller environments and assumed parallel training. Relative to a baseline approach, we showed training time reductions of 30.48% and 26.22% for two different simulated environments. The second of these works (McEnroe et al. [7]) examined reducing the inference latency of a DRL UAV obstacle avoidance model through model compression techniques. On two NVIDIA Jetson edge devices, we showed that relative to a baseline D3QN model, our proposed combined model compression technique reduces inference latency by an average of 45.90% with a negligible compromise in success rate. In contrast to these two works, in this work we explore both training time reduction and inference latency reduction. Additionally, we use completely different methods to that of [6] and [7]. Our method of reducing training time in this paper is transfer learning as opposed to ‘environment splitting’ and our method of reducing inference latency in this paper is state space reduction instead of model compression. Similar to [7], we evaluate our DRL obstacle avoidance models on a NVIDIA Jetson Nano edge device and a NVIDIA Jetson Orin Nano edge device. In this paper, however, we evaluate with two types of DRL algorithms (D3QN and SAC) as opposed to just one DRL algorithm (D3QN).

The key contributions of this paper are as follows:

- Analyze how state space reductions impact the UAV obstacle avoidance inference latency of DRL algorithms.
- Demonstrate transfer learning’s training time reduction benefits for DRL-based UAV obstacle avoidance.
- Propose a novel method (FERO) with open-sourced code¹ that enables efficient deployment of image-based DRL navigation algorithms on NVIDIA Jetson devices.

The remainder of the paper is structured as follows; Section II provides an overview of related research, Section III outlines background on DRL and the specific DRL algorithms used in the paper, Section IV outlines the details of our evaluation setup, Section V outlines our individual methods and combined approach, Section VI presents and analyses the results and Section VII concludes.

II. RELATED WORK

The method of UAV obstacle avoidance chosen in this paper is deep reinforcement learning (DRL). There are many other methods used to achieve UAV obstacle avoidance. For example, a commonly used method is trajectory optimization

(e.g., [8], [9]) where a collision-free path is computed that minimizes a specified cost (e.g., time) while satisfying various system constraints. We chose DRL as our obstacle avoidance approach because, despite its demonstrated effectiveness in UAV navigation, it remains comparatively underexplored relative to more traditional methods. Additionally, we evaluate performance in previously unseen environments. Traditional approaches such as trajectory optimization typically rely on precise environmental models, which can limit their adaptability and result in degraded performance when faced with novel environments. In contrast, when trained across an environment or environments with sufficient variability, DRL can generalize more robustly to unfamiliar conditions [10].

Our paper focuses on reducing the inference latency and training time of DRL UAV obstacle avoidance algorithms at the edge through state space reduction and transfer learning. Methods of reducing inference latency and training time of DRL algorithms and deep learning (DL) algorithms can overlap. As a result, with a particular focus on edge UAV navigation problems, we will split our related work discussion into two sections; ‘Reduce DL/DRL Inference Latency Through State Space Reduction’ and ‘Reduce DL/DRL Training Time Through Transfer Learning’.

A. REDUCE DL/DRL INFERENCE LATENCY THROUGH STATE SPACE REDUCTION

Numerous papers significantly reduce the input size of their DRL navigation state space before passing the input through the DRL models. Zhou et al. [11], Xue et al. [12] and Xue et al. [13] are examples of this. Zhou et al. [11] converts the front camera depth image of their UAV from a size of 256×144 to 100×20 pixels. They also encode the track angle between the UAV and target as a thick black line. Xue et al. [12] uses a VAE to convert an 84×84 UAV front camera depth image to a vector of length 50. This length 50 vector is used as part of the state space. Xue et al. [13] converts the front camera depth image of their UAV from a size of 128×72 pixels to a size of 16×16 pixels before processing through their DRL navigation algorithm. All these papers, however, do not explore the effect of reducing the state space on inference speed as they do not compare their inference speed results to a model where the state space was not significantly reduced.

The paper that has the most overlap with the reduced state space contribution of this paper is Shiri et al. [14]. Shiri et al. [14] introduces an effective language-guided RL method for autonomous systems with limited resources. Through a bar graph they demonstrate energy consumption improvement of a language-guided RL model when image input size is decreased. They also show that decreasing image size from 324×244 to 259×195 reduces the execution time of their RL model from 3000 ms to 550 ms. To the best of our knowledge, our paper is the first paper to examine how multiple image input reductions effect DRL inference latency of multiple algorithms for the problem of UAV autonomous navigation or UAV obstacle avoidance.

¹<https://github.com/PatMcE/Efficient-UAV-DRL-OA-Gym-Gazebo>

B. REDUCE DL/DRL TRAINING TIME THROUGH TRANSFER LEARNING

Zhu et al. [15] examines the application of DRL to mobile robot navigation where a laser ranging sensor is the chosen sensor. Through a reward vs episode graph they show that transfer learning can improve the convergence rate and reward convergence value in two Gazebo simulated test environments. Li et al. [16] presents a DRL and LIDAR based navigation method for mobile robots. Their method introduces an artificial potential field into the traditional reward function. They also examine how knowledge can be transferred from simple to complex environments in order to reduce training time. They evaluate the benefit of this knowledge transfer through reward vs episodes graphs and through recording training times in seconds.

Oriyama et al. [17] propose a human-in-the-loop transfer learning framework for autonomous robot collision avoidance. This framework uses human knowledge to pre-train models then refines in real-world environments with reinforcement learning. The benefit of transfer learning is primarily shown through loss function graphs. Loss function graphs show how during training the error between predicted actions and desired actions decreases over time. Gao et al. [18] present an incremental DRL training method for the path planning of a mobile robot. They transfer knowledge from a 2D environment to a 3D environment to improve training efficiency and model convergence. The positive affect of transfer learning on training time is primarily shown through a reward vs timestep graph. Gok et al. [19] propose a DRL approach for dynamic path planning of a mobile robot. They use transfer learning to improve training time and performance. The authors present cumulative reward vs episode graphs and average Q-value vs episode graphs for different stages of their approach. The benefit of transfer learning can be deduced if certain graphs are compared.

The paper that has the most overlap with the transfer learning contribution of this paper is Anwar et al. [20]. Anwar et al. [20] proposes a transfer learning based method for reducing the energy consumption and training time of a DRL-based UAV autonomous navigation model deployed on-board the UAV. The authors train in various simulated environments and then transfer this knowledge to test environments where the final few layers are fine-tuned. They present results for 3 transfer learning models where the final 2, 3 and 4 layers are fine-tuned/re-trained respectively. They also present results for a baseline model (referred to as E2E) where all layers are fine-tuned. In terms of training time reduction results, the authors present return vs episode graphs and the train time (in seconds) for their models. While Anwar et al. [20] presents training time results for multiple transfer learning models, it only applies this transfer learning to one DRL algorithm. The authors also fine-tune in test environments which means no test environment is unseen. Finally, the github associated with the paper is no longer being updated and relies on AirSim. Airsim will no longer be supported by Microsoft. Microsoft

are transitioning to Project AirSim, which is to be released under a commercial license [21].

Regarding the transfer learning contribution of our paper, in contrast to all the papers above, our paper shows the training time reduction benefit of transfer learning to a UAV DRL-based navigation problem where our approach is demonstrated for multiple distinct algorithms (D3QN and SAC). We apply transfer learning to a discrete action space value-based model (D3QN) and a continuous action space actor-critic model (SAC). Additionally, we demonstrate performance in unseen environments without fine-tuning in them and provide all the necessary code that is up to date as of the date of publication.

III. BACKGROUND

A. REINFORCEMENT LEARNING AND DEEP REINFORCEMENT LEARNING

As discussed in Section I, reinforcement learning (RL) is a subset of machine learning where an agent interacts with an environment until it can determine which of its actions will result in the most reward [2]. In discrete intervals called steps, the agent (e.g., UAV) uses observations and rewards from the environment to make decisions on what actions to take. These actions influence the subsequent observations and rewards, and the process repeats. A series of steps starting from an initial state (e.g., UAV at origin) up until a terminal state is called an episode. The episode enters its terminal state when the agent runs out of time, achieves its goal or meets some pre-defined condition (e.g., crash). The ultimate objective of the agent is to maximise the cumulative reward experienced each episode [22]. When introduced to large state/action spaces, RL algorithms can find learning from every state and determining an optimal reward path (i.e., policy) overwhelming. By combining RL with deep learning, deep reinforcement learning (DRL) can address such issues through leveraging neural networks.

B. DRL IMPLEMENTATION DETAILS

1) STATE SPACE

The state spaces we use throughout this paper (i.e., how the environment is represented to the UAV) is either 3 stacked images or a 1-D vector. In all but one set of experiments we use 3 stacked images, so we will focus on this state space (shown by Fig. 1). W and H in Fig. 1 changes for different experiments. The first image is a preprocessed raw depth image of what the UAV currently sees through its front facing depth camera (i.e., image from current timestep). The second image is the same as the first image but from the previous timestep. The third image contains all zeros except for the top left two values which represent the distance to the goal point in the x and y directions. We use multiple depth images so that movement direction can be inferred and we include goal point information as we found the algorithms worked much better when this information was included. The reason we

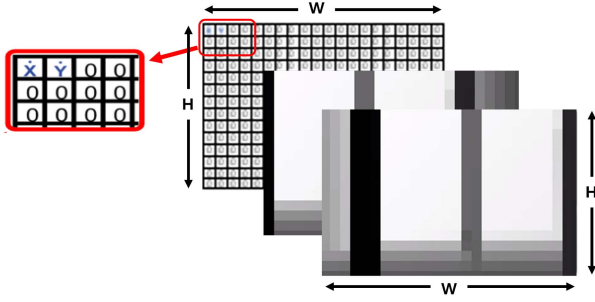


FIGURE 1. State space structure.

represented the goal point information with an image of all zeros except for the top left two values instead of just a vector of length two is that we found that our NVIDIA Jetson devices could process a 3 image state space much easier than a 2 image combined with a 1-D vector state space. This is because neural network architectures optimized for image processing on NVIDIA Jetson devices are typically designed to handle consistent spatial inputs (e.g., images). Combining image data with a 1-D vector requires additional preprocessing or custom layers, which can increase computational complexity and reduce efficiency.

2) ACTION SPACES

To test the generalizability and adaptability of our proposed approaches, we evaluate our approaches using a discrete action space DRL algorithm and a continuous action space DRL algorithm. In order to allow enough time for experimentation with the DRL algorithms we used relatively low dimension action spaces. We found that when we experimented with the same DRL algorithms using higher dimension action spaces much more training time was required to converge to an acceptable performance. The discrete action space we used is 3 actions (Forward, Left, Right), where each action moves the UAV forward/left/right for a fixed duration at 0.5 meters per second. The continuous action space used is 2 actions (vx, vy), where vx and vy are velocities in the x and y directions ranging from -0.5 to 0.5 meters per second. Algorithms that use this continuous action space move backwards as well as forward, left and right.

3) REWARD FUNCTION

We investigated four different reward functions. Given $\mathbb{CR}_{new}$ represents the new calculated cumulative reward, $\mathbb{CR}_{old}$ represents the previous cumulative reward, $\mathbb{D}_t$ the distance to goal point at the current timestep and $\mathbb{D}_{t-1}$ the distance to goal point at the previous timestep, equations 1, 2, 3 and 4 represent the four reward functions at all but the final timestep of each episode.

$$\mathbb{CR}_{new} = \mathbb{CR}_{old} - 1 + (\mathbb{D}_{t-1} - \mathbb{D}_t) \quad (1)$$

$$\mathbb{CR}_{new} = \mathbb{CR}_{old} - 5 + (\mathbb{D}_{t-1} - \mathbb{D}_t) \quad (2)$$

$$\mathbb{CR}_{new} = \begin{cases} 1 + (\mathbb{D}_{t-1} - \mathbb{D}_t), & \text{if } \mathbb{D}_t < \mathbb{D}_{t-1} \\ -1 + (\mathbb{D}_{t-1} - \mathbb{D}_t), & \text{if } \mathbb{D}_t \geq \mathbb{D}_{t-1} \end{cases} \quad (3)$$

$$\mathbb{CR}_{new} = \begin{cases} 5 + 5(\mathbb{D}_{t-1} - \mathbb{D}_t), & \text{if } \mathbb{D}_t < \mathbb{D}_{t-1} \\ -5 + 5(\mathbb{D}_{t-1} - \mathbb{D}_t), & \text{if } \mathbb{D}_t \geq \mathbb{D}_{t-1} \end{cases} \quad (4)$$

At the final timestep of each episode the cumulative reward for all reward functions is incremented by 100 if the UAV gets to the goal point (± 0.5 m) or is decremented by 100 if the UAV does not reach the goal point (± 0.5 m).

C. DISCRETE ACTION SPACE DRL ALGORITHM

We use two baseline DRL algorithms in this paper, the first is a discrete action space DRL algorithm, dueling double deep Q-network (D3QN). D3QN is a value-based DRL algorithm that is an improvement on the well-known deep Q-network (DQN) algorithm. DQN and DQN-based algorithms (e.g., D3QN) are well suited to problems with high-dimensional observation spaces (e.g., images) and small discrete action spaces [23]. Additionally, DQN-based algorithms were widely used for obstacle avoidance or path planning [24]. DQN and its variants are all off-policy and model-free algorithms.

The D3QN algorithm is summarized in Algorithm 1. The reward function used by the D3QN algorithm is the reward function defined by (1) as we found it performed best for D3QN models. In Algorithm 1, s_t denotes the state at time step t , a_t is the action selected at time t , and a represents a generic action from the action space. r_t is the reward received at time t , and s_{t+1} is the resulting next state. ϵ controls the exploration-exploitation trade-off, γ is the discount factor, $\mathcal{D}$ is the experience replay buffer, Q_θ is the online Q-network, and $Q'_{\theta'}$ is the target network. The variable done indicates whether the episode has terminated.

D. CONTINUOUS ACTION SPACE DRL ALGORITHM

The second baseline DRL algorithm that we employ in this paper is a continuous action space DRL algorithm, soft actor critic (SAC). SAC is an off-policy model-free actor-critic RL method known for its efficiency, stability and suitability to continuous action spaces [25]. Instead of focusing only on maximizing rewards, SAC incorporates entropy into its optimization strategy to balance exploration and exploitation. Entropy quantifies the amount of uncertainty in the current state and it can promote exploration through keeping the action selection process more random [26].

The key networks of a SAC algorithm are the actor network, critic networks and target networks. The actor network, also called the policy network, represents the stochastic policy $\pi(a | s)$. It outputs a probability distribution over actions for a given state. This probability distribution is a Gaussian distribution parameterized by mean values $\mu(s)$ and log-standard deviation values $\log \sigma(s)$. The two critic networks estimate the expected return (Q-value) of taking a specific action in a given state and then following the policy thereafter. The target networks are used to stabilise training, they are copies

Algorithm 1: D3QN Algorithm.

```

1: Initialize online network  $Q_\theta$  and target network  $Q'_{\theta'}$ 
2: Initialize replay buffer  $\mathcal{D}$ 
3: for episode = 1 to max_episodes do
4:   Reset the environment, get initial state  $s_0$ 
5:   Set episode_reward = 0, done = False
6:   while not done do
7:     Select action  $a_t$  according to  $\epsilon$ -greedy policy:

$$a_t = \begin{cases} \text{random,} & \text{probability } \epsilon \\ \arg \max_a Q_\theta(s_t, a), & \text{probability } 1 - \epsilon \end{cases}$$

8:     Execute action  $a_t$ , observe reward  $r_t$  and next state  $s_{t+1}$ 
9:     Store transition  $(s_t, a_t, r_t, s_{t+1}, \text{done})$  in  $\mathcal{D}$ 
10:    Sample random minibatch of transitions  $(s_j, a_j, r_j, s_{j+1}, \text{done}_j)$  from  $\mathcal{D}$ 
11:    Compute Double DQN target:

$$y_j = \begin{cases} r_j, & \text{if done}_j \\ r_j + \gamma Q'_{\theta'}(s_{j+1}, \arg \max_a Q_\theta(s_{j+1}, a)), & \text{otherwise} \end{cases}$$

12:    Update network using gradient descent to minimize prediction error
13:    Every  $X$  steps (e.g., 1000), copy the weights from  $Q_\theta$  to  $Q'_{\theta'}$ 
14:  end while
15: end for

```

of the critic networks but they have parameters that are slowly updated over time [27].

The steps of a SAC algorithm are summarized in Algorithm 2. The reward function used by the SAC algorithm is the reward function defined by (4) as we found it performed best for SAC models. In Algorithm 2, $s_t, a_t, a, r_t, s_{t+1}, \mathcal{D}$ and done are the same as in Algorithm 1. π_θ is the actor (policy) network parameterized by θ , and $Q_{\theta_1}, Q_{\theta_2}$ are the two critic (Q-value) networks with parameters θ_1 and θ_2 respectively. The corresponding target critic networks are represented as Q'_{θ_1} and Q'_{θ_2} . Finally, α is the temperature parameter controlling the entropy regularization term.

IV. EVALUATION SETUP

A. SIMULATION DETAILS

We train and test all our DRL obstacle avoidance models in 3D simulation. We choose Gazebo as our simulator and used a Gazebo UAV model² that replicates a real-life quadcopter UAV, the Uvify IFO-S. The Uvify IFO-S UAV uses a NVIDIA Jetson Nano as its on-board edge device. Like the real-life UAV, the Gazebo UAV model is integrated with a front facing Intel Realsense D435i depth camera. This camera produces

²https://github.com/decargroup/ifo_gazebo

Algorithm 2: Soft Actor Critic (SAC) Algorithm.

```

1: Initialize actor network  $\pi_\theta$ , critic networks  $Q_{\theta_1}, Q_{\theta_2}$ , and target networks  $Q'_{\theta_1}, Q'_{\theta_2}$ 
2: Initialize replay buffer  $\mathcal{D}$ 
3: for episode = 1 to max_episodes do
4:   Reset environment, get initial state  $s_0$ 
5:   Set episode_reward = 0, done = False
6:   while not done do
7:     Select action  $a_t \sim \pi_\theta(a_t | s_t)$ 
8:     Execute action  $a_t$ , observe reward  $r_t$  and next state  $s_{t+1}$ 
9:     Store transition  $(s_t, a_t, r_t, s_{t+1}, \text{done})$  in  $\mathcal{D}$ 
10:    Sample minibatch from  $\mathcal{D}$ 
11:    Compute target Q-value using critic target networks
12:    Update critic networks by minimizing loss between predicted and target Q-values
13:    Update actor network by maximizing expected reward and entropy
14:    If  $\alpha$  is not fixed, adjust temperature based on entropy
15:    Soft update target networks
16:  end while
17: end for

```

raw depth images which we use as part of the state space for our DRL algorithms.

To communicate with and train our UAV we used robot operating system (ROS) and Python. ROS is a versatile framework that supports the development of robot software. It offers crucial tools and libraries to simplify the creation of complex robotic applications by enabling modular communication between components (e.g., nodes and topics) [28]. To allow for easy training of RL models with ROS and Gazebo, we created a custom openAI gym environment. The custom gym environment allows us interface python files for training the DRL models with ROS topics from the UAV.

B. EVALUATION DETAILS

We use two Acer Nitro 5 laptops (NVIDIA GeForce GTX GPU, Intel Core i5 CPU) running Ubuntu 20.04, ROS Noetic and Gazebo to train all our models. We train using Pytorch and all training results are obtained on one of these two laptops. In order to get inference latency and other test results on the NVIDIA Jetson edge devices, we use the same evaluation setup as a previous paper (McEnroe et al. [7]). This setup is as shown in Fig. 2. The desktop computer runs the 3D simulated Gazebo environment and communicates via LAN with the NVIDIA Jetson Device (Nano or Orin Nano) which executes the DRL obstacle avoidance algorithms. The Gazebo simulated environment returns the sensor information over ROS topics that the DRL algorithm uses to generate actions.

When deploying the models on the NVIDIA Jetson devices it is not optimal to deploy Pytorch models. NVIDIA's

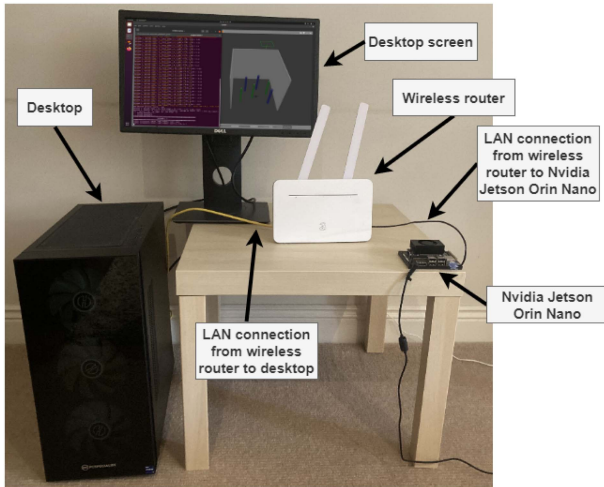


FIGURE 2. Evaluation setup for NVIDIA jetson orin nano (same for NVIDIA jetson nano).

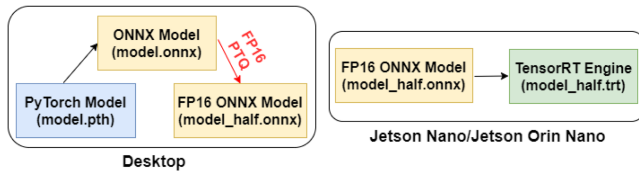


FIGURE 3. Pytorch model to TensorRT engine conversion.

TensorRT software allows conversion of standard model types (e.g., Pytorch or TensorFlow) into TensorRT engines. These TensorRT engines are optimized for the specific NVIDIA hardware on which they will be deployed. We convert from Pytorch to TensorRT as shown in Fig. 3. Note that ONNX, which stands for Open Neural Network Exchange, is an open-source standard that allows machine learning models to be transferred easily between different frameworks. Also note that before conversion to TensorRT we post-training quantize all our models (including the baseline models) from floating-point 32 precision to floating-point 16 precision. As proven by [7], this reduces inference latency while having minimal affect on success rate.

C. EVALUATION METRICS

We used multiple metrics to evaluate our proposed approaches. Bellow are the key metrics we record:

- **Learning Curve Plot** - Plot that visualizes average reward vs timesteps during training for one or more experiments. Each individual line/curve in the plot represents a single experiment and is called a learning curve. Average reward represents the mean total reward accumulated over the previous 100 episodes.
- **Min Reward Time (MRT)**: Training time to reach the minimum acceptable reward. Minimum acceptable reward is 25 for D3QN and 200 for SAC. We determined these two values by observing the approximate average reward over a 100 episode training period where the

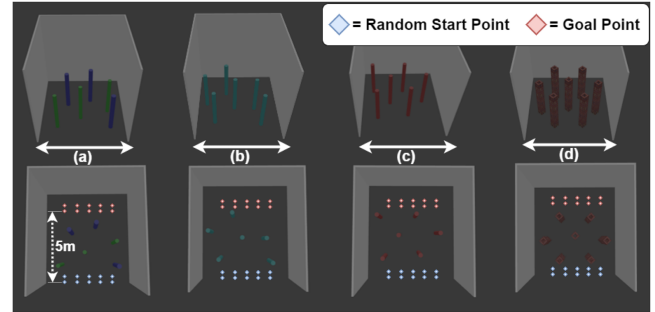


FIGURE 4. Evaluation environments: (a) Train environment (Env1), (b) 1st test environment (Env2), (c) 2nd test environment (Env3), and (d) Environment transfer learning pre-trained model trained in (Env0).

UAV was able to reach the goal point in at least 90% of training episodes. The values are different for the two algorithms because each algorithm uses a different reward function.

- **Best Reward**: Highest average reward reached during training.
- **Inference Latency**: The time it takes a model to processes the UAV's current state and determine the next action.
- **Success Rate (SR)**: Percentage of evaluation episodes where the UAV reaches the desired goal point.

D. ENVIRONMENTS

As there is no universally accepted 3D simulated environments used to evaluate UAV DRL navigation algorithms, we created our own Gazebo environments. These environments are particularly inspired by [29] and [30]. Fig. 4 shows the train environment, the two test environments and the environment the transfer learning pre-trained model is trained in. In both training and testing, the UAV randomly spawns at one of the random start points and aims to reach the goal point 5 meters directly in front of this start point.

V. IMPLEMENTATION DETAILS

In this paper we aim to produce well performing D3QN/SAC models while reducing DRL training time and inference latency as much as possible. We do this through state space reduction and transfer learning. We first look at the baseline model architectures, then explore determining the optimum state space, then outline how transfer learning can be applied to models with this optimum state space and finally we combine state space reduction and transfer learning in our combined approach (FERO).

A. MODEL ARCHITECTURES

The state space used by all but one model (when we try a flattened image state space) is as described in Section III-B1 and is depicted by Fig. 1. The D3QN and SAC model architectures that use this state space are as shown in Figs. 5 and 6 respectively. We modified the architectures of both the D3QN and SAC models such that we only pass the depth images through the convolutional layers. The final image just contains

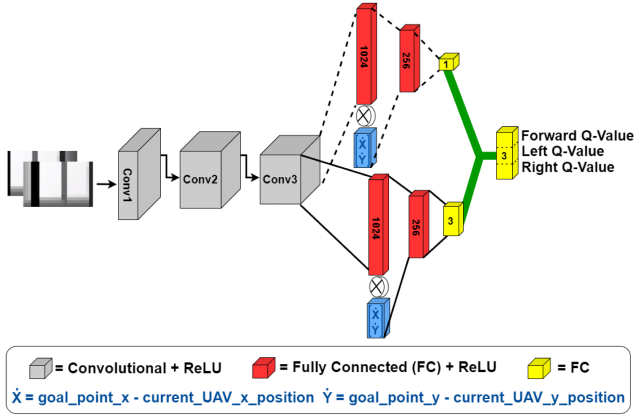


FIGURE 5. Modified D3QN network architecture.

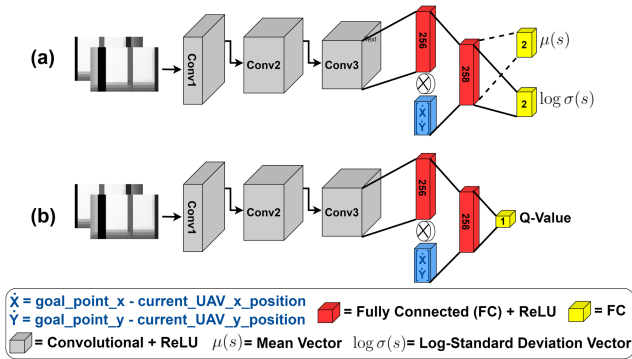


FIGURE 6. Modified network architectures of: (a) SAC actor and (b) SAC critic.

goal-point information which does not benefit from feature extraction so the goal-point information from this image is concatenated with the first fully connected layer. The convolutional layers change depending on the experiment but the architecture of the rest of the layers remains the same between experiments.

B. OPTIMUM STATE SPACE

We first looked at the performance of a D3QN and SAC model with a baseline state space. This baseline state space is of shape (3,80,106), the same as the state space we employed in a previous paper (McEnroe et al. [7]). The state space is as in Fig. 1 (where H is 80 pixels and W is 106 pixels). A raw depth image returned by the UAV is 480 pixels by 640 pixels, so there is a bit of pre-processing done to the original image but it still remains unnecessarily large.

Next we next looked at finding the smallest usable state space of the form (3,H,W) where H and W are kept in the same ratio as the original raw depth image returned by the UAV (i.e., 480:640 pixels). We needed to ensure that the D3QN/SAC models with the smaller state space shape would retain the same success rate as the D3QN/SAC models with the baseline state space. We found that by decreasing the number of convolutional layers from 3 to 2 we could use smaller images. Through trying various combinations of small state spaces of

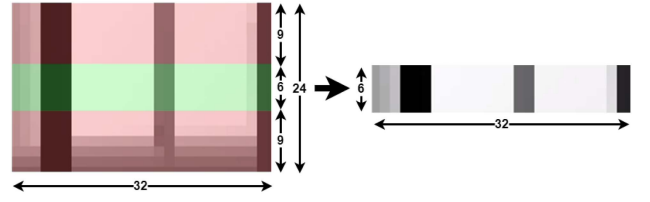


FIGURE 7. Image cropping for (3,6,32) state space.

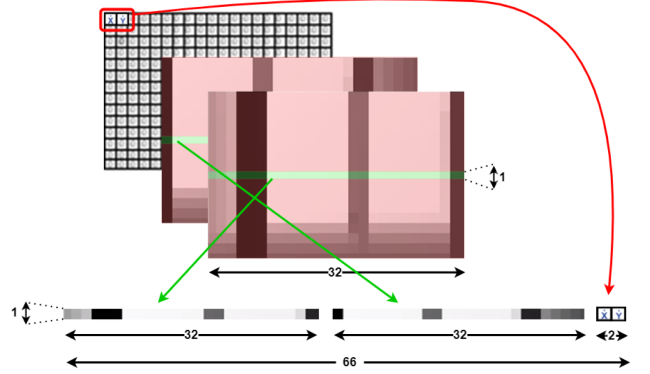


FIGURE 8. Generation of flattened state space.

the form (3,H,W) and combinations of filter/kernel sizes and stride values for the 2 convolutional layers we determined that a state space of shape (3,24,32) could be used for both D3QN and SAC models without an effect on performance. We tried state spaces smaller than (3,24,32) and increased the state space size until performance was maintained. 24 and 32 represent a 20 fold decrease on the original height/width of 480 and 640. This is in contrast to an approximately 6 fold decrease on the original height/width that the baseline state space achieves.

We also realized that since the obstacles we are avoiding are all cylinders/pylons, the top and bottom parts of the image are not adding any new information compared to the middle part of the image. As shown by Fig. 7, we cropped the top and bottom of the image such that the new shape of the state space is (3,6,32). This is the second state space we examine. If the height is decreased below 6 pixels (to 5) the output of the 2nd convolutional layer has a height of 0 and thus 6 pixels is the smallest height we can use.

Finally, as shown in Fig. 8, we explored a state space where we completely flattened the two depth images and where the goal point information was concatenated to the end of the flattened images. When evaluating this state space we completely removed the convolutional layers of both models so that there would not be the issue of height 0 outputs from convolutional layers.

C. TRANSFER LEARNING

Transfer learning (TL) is a form of machine learning where previously trained neural network weights can be transferred to new training scenarios in order to reduce convergence time

TABLE 1. Optimum Hyperparameter Values for D3QN and SAC

Hyperparameter	D3QN Value	SAC Value
Learning Rate ^b	0.00025	3e-4
Batch Size	64	256
Discount Factor	0.99	0.99
Target Network Update Frequency	1000	N/A
Replay Memory Size	100000	50000
Initial Epsilon	1	N/A
Final Epsilon	0.05	N/A
Epsilon Decay Steps ^a	75000	N/A
Softupdate Factor	N/A	1e-2
Entropy Temperature	N/A	0.3

^a Amount of steps it takes to go from initial to final epsilon value.

^b For SAC, this is the actor and critic learning rate.

and save on resources [19], [31]. For each algorithm (D3QN and SAC) we trained a model in ‘Env0’ from Fig. 4(d). We used these ‘pre-trained’ models as a starting point when re-training (i.e., fine-tuning) new models in the train environment (‘Env1’ from Fig. 4(a)). We fine-tuned the Env0 pre-trained models in Env1 with no layers frozen, just the first convolutional layer frozen, just the first two convolutional layers frozen and just the first two convolutional layers and the first fully connected layer frozen.

Relative to models where we trained in ‘Env1’ from scratch, the fine-tuned models (i.e., TL models) required significantly less training time. All our TL models fine-tuned in ‘Env1’ are also evaluated in completely unseen environments (‘Env2’ and ‘Env3’).

D. FERRO

FERRO stands for **efficient edge deep reinforcement learning based obstacle avoidance**. This is our novel approach that combines state space reduction, transfer learning and TensorRT conversion in order to improve image-based DRL obstacle avoidance efficiency. FERRO is defined by Algorithm 3. When training the first time using FERRO, no model is trained in a new environment. We are first required to determine the optimum state space dimensions and train a model that can be used for transfer learning later. When training the second time using FERRO, a low inference latency NVIDIA Jetson deployable model can be trained in a new environment but the training time is high as time is required to determine the best combination of transfer learning layers to freeze. The third and remaining times using FERRO, the full benefit is seen where a low inference latency NVIDIA Jetson deployable model can be trained in a new environment in a short amount of time.

VI. RESULTS AND ANALYSIS

A. STATE SPACE AND CONVOLUTION LAYER REDUCTION RESULTS

Table 2 summarizes the results of reducing state space dimensions and convolutional layers. All D3QN models are

Algorithm 3: FERRO.

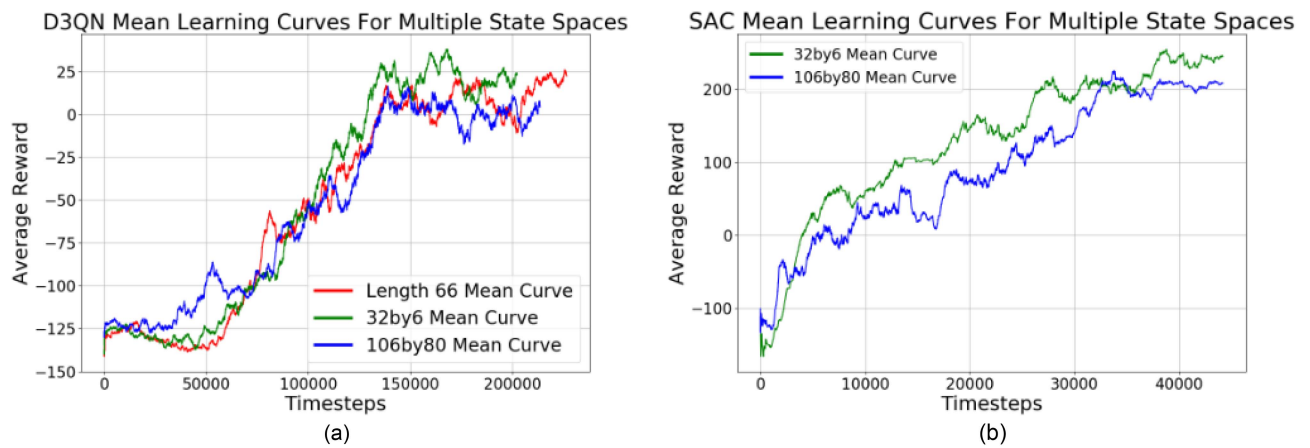
- 1: **Input:** Simulated training environment $\mathcal{E}$, usage count n
- 2: **Output:** Low inference latency DRL model for deployment on NVIDIA Jetson edge device
- 3: **Assumptions:** DRL model state space consists of just images of shape (C, H, W) , where C is the number of channels, H is height, and W is width. Also assume $\mathcal{E}$ is different every time FERRO is used.
- 4: **if** FERRO is used for the first time ($n = 1$) **then**
- 5: Through experimentation determine smallest values of H & W that retain success rate of a baseline state space (e.g., state space without pre-processing). Keep C the same and change convolutional layers if needed.
- 6: For future transfer learning, train a base DRL model using this optimum state space and optimum set of convolutional layers.
- 7: **else if** FERRO is used for the second time ($n = 2$) **then**
- 8: Load the pre-trained base model
- 9: With the optimum state space/convolutional layers, fine-tune multiple models for $\mathcal{E}$. Each model should have a different set of layers frozen before fine-tuning starts (e.g., no layers frozen, just 1st convolutional layer frozen, just 1st and 2nd convolutional layer frozen, etc).
- 10: Note the best set of layers to freeze and convert the best performing trained model to TensorRT for edge deployment (see Fig. 3)
- 11: **else**
- 12: With the optimum state space/convolutional layers, apply transfer learning to the pre-trained base model using the previously determined optimal layer freeze configuration
- 13: Convert this model to TensorRT for edge deployment (see Fig. 3)
- 14: **end if**

trained for 5000 episodes and all SAC models are trained for 2000 episodes. To increase the accuracy of results and to minimize noise, the results in each row of Table 2 and the learning curves of each learning curve plot are obtained through running three experiments with the same conditions and taking the average (i.e., mean) result. We did this as single experimental runs can exhibit high variability due to random factors. Additionally, smoothing is applied to all learning curves where we apply a running average with a window size of 100 timesteps. This is done to reduce frequent fluctuations and help reveal overall trends.

Fig. 9(a) and (b) compares the mean learning curves for differing state space shapes for the D3QN and SAC algorithms respectively. SAC does not come close to convergence for a state space of (1,66) so this state space is not included in Table 2 or Fig. 9(b). Using hyperparameter tuning we found

TABLE 2. Effect of Reducing State Space Dimensions and Convolutional Layers on Minimum Reward Time (MRT), Best Reward Value, Inference Latency and Success Rate (SR) in Seen (Env1) and Unseen (Env2&3) Environments. Inference Latency and Success Rate Results Shown on NVIDIA Jetson Orin Nano and (NVIDIA Jeston Nano)

Configuration	MRT ^d (hours)		Best Reward ^d		Inference Latency ^{de} (ms)		Env1 SR ^d (%)		Env2&3 SR ^{df} (%)	
State Space Shape	D3QN	SAC	D3QN	SAC	D3QN	SAC	D3QN	SAC	D3QN	SAC
(3, 80, 106) ^a	failed ^g	17.35	20.39	237.14	2.71 (7.86)	2.61 (7.75)	94 (92)	90 (89)	89.5 (88)	86.5 (88)
(3, 6, 32) ^b	57.63	15.89	38.49	254.87	1.61 (4.59)	2.49 (7.66)	96 (95)	91 (92)	91 (89)	88 (88.5)
(1, 66) ^c	61.34	N/A	37.59	N/A	1.24 (3.49)	N/A	95 (96)	N/A	69 (68.5)	N/A

^a with convolutional layers [(out/kernel/stride)]: [(32/8x8/4), (64/4x4/2), (64/3x3/1)]^b with convolutional layers [(out/kernel/stride)]: [(32/4x3/2), (64/2x2/1)] ^c with no convolutional layers ^d As defined in Section IV.C^e Inference latency results averaged over all environments (Env1, Env2 and Env3) ^f Env2 & Env3 results averaged^g D3QN (3,80,106) models do not reach minimum reward value threshold for all experiments**FIGURE 9.** (a) D3QN and (b) SAC Mean learning curves for state space and convolutional layer combinations discussed in Table 2.

the best performing hyperparameters to be as described in Table 1.

B. STATE SPACE AND CONVOLUTION LAYER REDUCTION RESULTS ANALYSIS

The results of our state space and convolutional layer reduction experiments are shown in Table 2 and Fig. 9. The green row of Table 2 relative to the row of the (3,80,106) baseline state space in the same table clearly shows that state space and convolutional layer reduction can significantly reduce DRL obstacle avoidance inference latency while slightly improving success rates. Interestingly, the D3QN inference latency is reduced significantly more by state space and convolutional layer reduction than the SAC inference latency. We speculate that this discrepancy could be because D3QN relies more on convolutional layers for feature extraction, so state space and convolutional layer reduction directly impacts its computational load and inference time more. Additionally, SAC's architecture includes additional components like stochastic sampling, which might dominate its inference latency.

The (1,66) state space row of Table 2 does not include results for SAC as we could not train a model that came close to convergence for this state space. However, from the D3QN results of this state space we can see that there is a further

reduction in inference latency beyond the (3,6,32) state space inference latency. This (1,66) state space does, however, struggle with performance in unseen environments for D3QN as can be seen from the Env2&3 success rate column of Table 2.

Regarding the training time results of state space and convolutional layer reduction shown by Table 2 and Fig. 9, we can see that a state space of (3,6,32) again outperforms the baseline (3,80,106) state space. This is particularly evident through the reduced Minimum Reward Time (MRT) values of the (3,6,32) state space relative to the (3,80,106) state space in Table 2. The D3QN models with the (3,80,106) state space failed to converge due to two of the three experiments not quite reaching the minimum reward threshold of 25. This could be because the larger (3,80,106) state space results in increased input complexity which leads to slower and less stable learning. The learning curves of Fig. 9 also visually demonstrate the slightly higher convergence values achieved by the (3,6,32) state space relative to the (3,80,106) state space. As one would expect, the effect of state space reduction on training time is not as significant as the affect on inference latency. Additionally, similar to the inference latency results, the positive effect of the state space reduction has more of an effect on the D3QN training time than the SAC training time.

TABLE 3. Effect of Transfer Learning (TL) on Minimum Reward Time and Best Reward Value

State Space Shape	Transfer Learning (TL) Applied	Min Reward Time (hours) [and Timesteps] ^a		Best Reward ^a	
		D3QN	SAC	D3QN	SAC
(3, 80, 106)	No TL ^b	failed	17.35 [34,619]	20.39	237.14
(3, 6, 32)	No TL ^c	57.63 [131,413]	15.89 [30,623]	38.49	254.87
	no layers frozen	35.58 [83,611]	13.59 [26,115]	33.82	255.13
	conv1 ^d frozen	37.06 [86,010]	9.11 [18,691]	38.61	247.26
	conv1, conv2 frozen	25.17 [60,318]	7.12 [13,818]	38.74	254.32
	conv1, conv2, fc1 ^e frozen	N/A	11.56 [23,863]	-5.33	221.93

^a As defined in Section IV.C ^b Same as first row of Table 2 ^c Same as green row of Table 2 ^d conv1 = first convolutional layer

^e fc1 = first fully connected layer

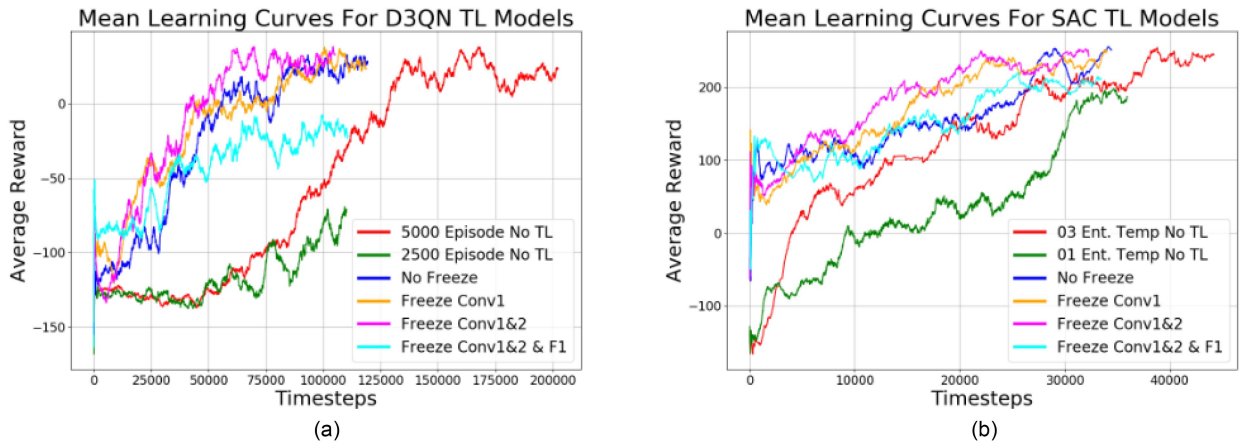


FIGURE 10. Mean learning curves for (a) D3QN and (b) SAC TL models discussed in Table 3.

C. TRANSFER LEARNING RESULTS

Table 3 summarizes the results of different transfer learning models where D3QN transfer learning models are ran for 2500 episodes and SAC transfer learning models are ran for 1500 episodes. All transfer learning models in Table 3 use the best performing state space shape and convolutional layer combination from Table 2 (shown by green row of Table 2). We found D3QN transfer learning models best performed with all hyperparameters the same as Table 1 except for learning rate set to 0.000025 instead of 0.00025 and epsilon steps set to 35000 instead of 75000. We found SAC transfer learning models best performed with all hyperparameters the same as Table 1 except for learning rate set to 3e-5 instead of 3e-4 and entropy temperature set to 0.1 instead of 0.3. We determined the transfer learning model hyperparameters for both algorithms through extensive grid search hyperparameter tuning where many different values for each hyperparameter was tested (e.g., epsilon steps of 15000, 35000, 50000, 75000 and 100000 was tested). As with Table 2, all transfer learning results are obtained through running three experiments and taking the average (i.e., mean) result.

Fig. 10 shows the mean learning curves for each transfer learning model discussed in Table 3. For the sake of comparison, all these figures include the model shown by the yellow row of Table 3 (and green row of Table 2) where

no transfer learning is applied (red curve in Fig. 10(a) and (b)). Each of these figures also show a second ‘no transfer learning’ model (green curve in Fig. 10(a) and (b)) that is trained from scratch and ran for the same amount of episodes as the transfer learning models. These green curve models keep their hyperparameters the same as the transfer learning models except the D3QN model starts with an epsilon value of one. We included the models shown by the red and green learning curves to emphasize that transfer learning enables convergence in a lower amount of timesteps.

D. TRANSFER LEARNING RESULTS ANALYSIS

The impact of transfer learning (TL) on training time is depicted with Table 3 and Fig. 10. This table and figure shows that, relative to a baseline model trained from scratch (yellow row Table 3), fine-tuning a DRL algorithm (last 4 rows of Table 3) significantly reduces the minimum reward time while retaining very similar best reward results. We found the TL model where both convolutional layers were frozen to have performed particularly well. This is not a surprising result as it makes sense that models trained in 2 different environments but with similar aesthetics are more likely to share similar feature extraction (i.e., convolutional layer) weights and biases than weights and biases for the rest of their networks. The green row of Table 3 shows that this case where both

TABLE 4. Comparison of Baseline and FERO Model Inference Latency and Success Rate on NVIDIA Jetson Orin Nano and (NVIDIA Jetson Nano)

Model	Inf. Latency ^a (ms)		Success Rate ^b (%)	
	D3QN	SAC	D3QN	SAC
Baseline	2.71(7.86)	2.61(7.75)	91(89.33)	87.67(88.33)
FERO	1.61 (4.59)	2.49 (7.66)	94.33(93.67)	89.67(90.67)

^a Average inference latency over Env1, Env2 and Env3^b Average success rate over Env1, Env2 and Env3

convolutional layers were frozen outperforms the minimum reward time results of all other TL models for both algorithms. The pink learning curves of Fig. 10 also demonstrate the superior training of this case where both convolutional layers were frozen. Finally, it is interesting to note that the freezing of the second convolutional layer particularly benefited the D3QN models.

E. FERO RESULTS AND ANALYSIS

The training results of using FERO are the same as the training results of the TL models where both convolutional layers are frozen. These results are shown through the green row of Table 3 and the pink curves in Fig. 10. The green row of Table 3 relative to the first row of Table 3 shows that FERO considerably improves minimum reward time and best reward for both D3QN and SAC algorithms. For D3QN, our approach improves best reward from 20.39 to 38.74 and allows valid minimum reward time results (i.e., not to fail). For SAC, our best reward improves from 237.14 to 254.32 and the minimum reward time decreases by 58.96%. Additionally, Table 4 shows that relative to the baseline, FERO considerably improves inference latency and has a positive effect on success rate. For D3QN, FERO improves success rate while reducing inference latency by 40.59% and 41.60% on the Orin Nano and Nano respectively. For SAC, FERO improves success rate while reducing inference latency by 4.60% and 1.16% on the Orin Nano and Nano respectively.

VII. CONCLUSION

This paper explores methods to reduce training time and inference latency in DRL-based UAV obstacle avoidance algorithms. We first analyzed the impact of different state space shapes/sizes, particularly focusing on the impact on inference latency. We then demonstrated the advantages of transfer learning in reducing training time through learning curves and metrics such as the minimum reward time. To ensure the generalizability of our findings, we evaluated our approaches using two DRL algorithms (D3QN and SAC) across two edge devices (NVIDIA Jetson Nano and NVIDIA Jetson Orin Nano). Finally, we introduced a novel combined method, FERO, which integrates state space reduction, transfer learning, and TensorRT optimization. This method facilitates the efficient deployment of image-based DRL obstacle avoidance algorithms on NVIDIA Jetson platforms. Our results, encompassing both individual techniques and the combined

approach, underscore the significant benefits of state space reduction and transfer learning in enhancing the efficiency of UAV DRL obstacle avoidance algorithms on edge devices. While our FERO evaluations were conducted in static environments, the method could be adapted to dynamic scenarios involving moving obstacles. Future work could address dynamic scenarios, where the introduction of moving obstacles may require the need for faster inference latency and for trade-offs to be made between inference speed and performance metrics such as success rate.

REFERENCES

- [1] P. McEnroe, S. Wang, and M. Liyanage, "A survey on the convergence of edge computing and AI for UAVs: Opportunities and challenges," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15435–15459, Sep. 2022.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [3] Y. Li, H. Fang, M. Li, Y. Ma, and Q. Qiu, "Neural network pruning and fast training for DRL-based UAV trajectory planning," in *Proc. 27th IEEE Asia South Pacific Des. Automat. Conf.*, 2022, pp. 574–579.
- [4] L. Xie, S. Wang, S. Rosa, A. Markham, and N. Trigoni, "Learning with training wheels: Speeding up training with a simple controller for deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 6276–6283.
- [5] E. Faniadis and A. Amanatiadis, "Deep learning inference at the edge for mobile and aerial robotics," in *Proc. IEEE Int. Symp. Saf., Secur., Rescue Robot.*, 2020, pp. 334–340.
- [6] P. McEnroe, S. Wang, and M. Liyanage, "Towards faster DRL training: An edge AI approach for UAV obstacle avoidance by splitting complex environments," in *Proc. IEEE 21st Consum. Commun. Netw. Conf.*, 2024, pp. 396–399.
- [7] P. McEnroe, S. Wang, and M. Liyanage, "Towards latency efficient DRL inference: Improving UAV obstacle avoidance at the edge through model compression," in *Proc. IEEE 27th Int. Conf. Intell. Transp. Syst.*, 2024, pp. 4242–4249.
- [8] Z. Liu, S. Wen, G. Huang, S. Li, and Z. Deng, "Agricultural UAV obstacle avoidance system based on a depth image inverse projection algorithm and B-spline curve trajectory optimization algorithm," *Inf. Technol. Control*, vol. 53, no. 3, pp. 736–757, 2024.
- [9] M. Lu et al., "HEPP: Hyper-efficient perception and planning for high-speed obstacle avoidance of UAVs," 2025, *arXiv:2505.17438*.
- [10] S. Garg, H. Masnavi, B. Fidan, and F. Janabi-Sharifi, "Dual agent learning based aerial trajectory tracking," *IEEE Robot. Autom. Lett.*, vol. 10, no. 6, pp. 5609–5616, Jun. 2025.
- [11] B. Zhou, W. Wang, Z. Liu, and J. Wang, "Vision-based navigation of UAV with continuous action space using deep reinforcement learning," in *Proc. Chin. Control Decis. Conf.*, 2019, pp. 5030–5035.
- [12] Y. Xue and W. Chen, "Combining motion planner and deep reinforcement learning for UAV navigation in unknown environment," *IEEE Robot. Autom. Lett.*, vol. 9, no. 1, pp. 635–642, Jan. 2024.
- [13] Z. Xue and T. Gonsalves, "Vision based drone obstacle avoidance by deep reinforcement learning," *AI*, vol. 2, no. 3, pp. 366–380, 2021.
- [14] A. Shiri, M. Navardi, T. Manjunath, N. R. Waytowich, and T. Mohsenin, "Efficient language-guided reinforcement learning for resource-constrained autonomous systems," *IEEE Micro*, vol. 42, no. 6, pp. 107–114, Nov/Dec. 2022.
- [15] J. Zhu, C. Yang, Z. Liu, and C. Yang, "Path planning of mobile robot based on deep reinforcement learning with transfer learning strategy," in *Proc. IEEE 37th Youth Academic Annu. Conf. Chin. Assoc. Automat.*, 2022, pp. 1242–1246.
- [16] W. Li, M. Yue, J. Shangguan, and Y. Jin, "Navigation of mobile robots based on deep reinforcement learning: Reward function optimization and knowledge transfer," *Int. J. Control, Automat. Syst.*, vol. 21, no. 2, pp. 563–574, 2023.
- [17] M. Oriyama, P. Hartono, and H. Sawada, "Human-in-the-loop transfer learning in collision avoidance of autonomous robots," *Biomimetic Intell. Robot.*, vol. 5, 2025, Art. no. 100215.
- [18] J. Gao, W. Ye, J. Guo, and Z. Li, "Deep reinforcement learning for indoor mobile robot path planning," *Sensors*, vol. 20, no. 19, 2020, Art. no. 5493.

- [19] M. Gök, “Dynamic path planning via dueling double deep Q-network (D3QN) with prioritized experience replay,” *Appl. Soft Comput.*, vol. 158, 2024, Art. no. 111503.
- [20] A. Anwar and A. Raychowdhury, “Autonomous navigation via deep reinforcement learning for resource constraint edge nodes using transfer learning,” *IEEE Access*, vol. 8, pp. 26549–26560, 2020.
- [21] C. A. Dimmig et al., “Survey of simulators for aerial robots: An overview and in-depth systematic comparisons,” *IEEE Robot. Autom. Mag.*, vol. 32, no. 2, pp. 153–166, Jun. 2025.
- [22] E. Cetin, C. Barrado, G. Muñoz, M. Macias, and E. Pastor, “Drone navigation and avoidance of obstacles through deep reinforcement learning,” in *Proc. IEEE/AIAA 38th Digit. Avionics Syst. Conf.*, 2019, pp. 1–7.
- [23] K. Zhu and T. Zhang, “Deep reinforcement learning based mobile robot navigation: A review,” *Tsinghua Sci. Technol.*, vol. 26, no. 5, pp. 674–691, 2021.
- [24] C. Yan, X. Xiang, and C. Wang, “Towards real-time path planning through deep reinforcement learning for a UAV in dynamic environments,” *J. Intell. Robot. Syst.*, vol. 98, pp. 297–309, 2020.
- [25] T. Haarnoja et al., “Soft actor-critic algorithms and applications,” 2018, *arXiv:1812.05905*.
- [26] B. B. Elallid, N. Benamar, M. Bagaa, and Y. Hadjadj-Aoul, “Enhancing autonomous driving navigation using soft actor-critic,” *Future Internet*, vol. 16, no. 7, 2024, Art. no. 238.
- [27] J. C. de Jesus, V. A. Kich, A. H. Kolling, R. B. Grando, M. A. D. S. L. Cuadros, and D. F. T. Gamarra, “Soft actor-critic for navigation of mobile robots,” *J. Intell. Robot. Syst.*, vol. 102, no. 2, 2021, Art. no. 31.
- [28] M. Quigley et al., “ROS: An open-source robot operating system,” in *Proc. ICRA Workshop Open Source Softw.*, Kobe, Japan, 2009, pp. 1–3.
- [29] S. Zhang, Y. Li, and Q. Dong, “Autonomous navigation of UAV in multi-obstacle environments based on a deep reinforcement learning approach,” *Appl. Soft Comput.*, vol. 115, 2022, Art. no. 108194.
- [30] Y. Zhang, K. H. Low, and C. Lyu, “Partially-observable monocular autonomous navigation for UAV through deep reinforcement learning,” in *Proc. AIAA AVIATION 2023 Forum*, 2023.
- [31] O. Bouhamed, H. Ghazzai, H. Besbes, and Y. Massoud, “Autonomous UAV navigation: A DDPG-based deep reinforcement learning approach,” in *Proc. IEEE Int. Symp. Circuits Syst.*, 2020, pp. 1–5.



PATRICK McENROE (Associate Member, IEEE) received the M.Eng. degree in electronic and computer engineering and the Ph.D. degree in computer science from University College Dublin, Dublin, Ireland, in 2020 and 2025, respectively. His research interests include deep reinforcement learning, edge AI, UAVs, and autonomous navigation.



privacy for mobile networks.

SHEN WANG (Senior Member, IEEE) received the Ph.D. degree from Dublin City University, Dublin, Ireland, in 2016. He is currently an Assistant Professor with the School of Computer Science, University College Dublin, Dublin. Dr. Wang has been involved with several EU projects as a co-PI, WP, and Task leader in big trajectory data streaming for air traffic control and trustworthy AI for intelligent cybersecurity systems. His research interests include connected autonomous vehicles, deep reinforcement learning, security and



MADHUSANKA LIYANAGE (Senior Member, IEEE) received the Doctor of Technology degree in communication engineering from the University of Oulu, Oulu, Finland, in 2016. He is currently an Associate Professor, Ad Astra Fellow, and the Director of the Network Softwarization and Security Labs with the School of Computer Science, University College Dublin, Dublin, Ireland. He is also a Docent/Adjunct Professor with the Centre for Wireless Communications, University of Oulu, and an Honorary Adjunct Professor with the University of Ruhuna, Matara, Sri Lanka, and with the University of Sri Jayawardhanapura, Nugegoda, Sri Lanka. His research interests include 5G/6G, blockchain, network security, artificial intelligence, explainable AI, federated learning, network slicing, Internet of Things, and multi-access edge computing. He was a recipient of the prestigious Marie Skłodowska-Curie Actions Individual Fellowship and the Government of Ireland Postdoctoral Fellowship from 2018 to 2020. In 2021, 2022, 2023, and 2024, he was ranked among the World’s top 2% scientists from 2020 to 2024 in the List prepared by Elsevier BV and Stanford University, USA. He was awarded the Irish Research Council (IRC) Research Ally Prize as part of the IRC Researcher of the Year 2021 awards for the positive impact he has had as a supervisor, “2022 Tom Brazil Excellence in Research Award” from the SFI CONNECT Centre, and the “2020 IEEE ComSoc Outstanding Young Researcher” award from IEEE ComSoc EMEA in 2020. More info: www.madhusanka.com